



National University of Singapore

EE4002R CA3 Report:

Deep Learning for LDPC Code Decoding in Digital Communications

Academic Year 2024/2025 Semester 2

Supervisor: Dr Zhang Jianwen

Student Name	Matriculation Number
Leong Kah Sheng	A0233541N

1. Abstract	2
2. Acknowledgements	2
3. Introduction	3
3.1 Implementation of BP algorithm	6
4. Methodology	7
4.1 Model Architecture	8
4.1.1 Overview	8
4.2 Core Components	9
4.2.1 Base Matrix and expansion	9
4.2.2 Shared Weights	10
4.2.3 Residual Connections	10
4.3 Message Passing Logic	12
4.3.1 Variable Node Update	12
4.3.2 Check Node Update:	13
4.3.3 Posterior LLR Computation	14
4.3.4 Decoding Flow Diagram	15
4.4 Training Framework	16
4.4.1 Data Generation and Preparation	18
4.4.2 Model Architecture	18
3.4.3 Training Configuration	18
3.4.4 Training Loop Details	19
3.4.5 Loss Computation	19
3.4.8 Visualisation	19
4. Results and Analysis	20
4. 1 Performance Metrics	20
4.2 Performance Graphs and Results:	20
5.1 Advantages	21
6. Future Works	21
7. Conclusion	22
8. References	23
9. Appendix	24

1. Abstract

This paper presents a neural decoder architecture, which we named ResidualWeightSharingDecoder, for decoding 5G Low-Density Parity-Check (LDPC) codes. The model utilises the structured nature of quasi-cyclic LDPC codes (also known as a base graph) by expanding it into a full form parity check matrix. The base graph enables efficient parameter sharing for messages (in the expanded graph) that belong to the same group, which is determined by entries of the base graph. In addition, unlike traditional belief propagation (BP), this decoder integrates residual connections to enhance convergence and accuracy over multiple iterations. Residual learning allows the model to reuse information from previous iterations.

Our experimental results indicate that the model can generalise well across codewords while preserving decoding structure, making it a promising direction for future neural error-correction systems.

2. Acknowledgements

I would like to express my deepest gratitude to my supervisor, Professor Zhang Jianwen, for his invaluable guidance, support, and encouragement throughout the course of this Final Year Project. His patience and deep knowledge have been instrumental in shaping both the direction, execution and understanding of my work.

I would also like to thank my family for their unwavering support and understanding during this period. Their belief in me has been a constant source of motivation.

A special thank you to my girlfriend, whose encouragement, emotional support, and constant reassurance have helped me stay focused and resilient throughout this journey.

This project would not have been possible without the contributions and support of each of you. Thank you.

3. Introduction

Error correction is essential in digital communication to ensure that data transmitted across noisy channels remains accurate and reliable. Without error correction, noise and interference in the communication channel can lead to data corruption, which reduces performance metrics such as utilisation rate, and

increases the retransmission rates of the communication link. Low-Density Parity-Check (LDPC) codes are one of the most effective error-correcting codes widely used in high-performance applications, including satellite communications, 5G networks, and data storage. LDPC codes are structured using a sparse parity-check matrix that allows efficient error correction through algorithms like belief propagation (BP). Accurate decoding of these codes is critical because even small improvements in Bit Error Rate (BER) can lead to significant enhancements in communication reliability and quality.

Deep learning, also known as neural networks with multiple layers, provides a promising alternative for LDPC decoding due to the similarities of a tanner graph, which when unfolded resembles a sparse (specific and fewer connections than usual neural networks). Thus, it is likely that a neural network model is able to learn weights that can optimise the decoding process, which implies being adaptable to different noise levels and capable of generalising across diverse channel conditions.

The goal of this report is to apply deep learning, to the task of LDPC decoding and to assess the performance of this approach in comparison to traditional decoding methods like belief propagation (BP). The similarities between a Tanner graph and a neural network offers a worthy exploration of training a neural network-based decoder, which can help verify if deep learning based decoders can achieve similar performance rates (measured by BER and FER) with lesser computation resources.

Main Contributions:

1. Benchmarking Traditional Decoding: We implemented and tested the message-passing algorithm (belief propagation) using a parity-check matrix from David McKay's website. This involved BPSK modulation of the codewords, simulating Gaussian white noise, and plotting BER vs. SNR performance to establish a baseline.

2. Development of a Deep Learning-based Decoder: A Neural Network utilises the structure of a tanner graph, which uses weight sharing and residual connections on top of the standard tanner graph structure. The NN model was trained, validated and tested for its performance to observe the capabilities to decode short codewords with different noise patterns.

3. Verification of a similar literature: As a research paper was referenced to implement the weight sharing and residual decoder, we are also able to verify their results and observe the said performance in the paper with our implementation.

Overview of LDPC Codes

LDPC codes are a type of linear block code designed for error correction in digital communication, particularly in noisy environments. They are defined by a matrix, H . Specifically, this matrix H is known as a sparse parity-check matrix. “Sparse” refers to the matrix having more zeros than ones in each row and column, making them “low-density” or “sparse.” This matrix is the key to encoding and decoding processes, as it determines the relationship between the bits that are being sent. The parity refers to an error-detection method to ensure that data is transmitted accurately. It involves adding an extra bit, called the parity (or check) bit, to a group of data bits. There are two main types of parity:

- Even Parity: The parity bit is set so that the total number of 1s in the data (including the parity bit) is even. For example, if the data is `1011` (which has three 1s, an odd number), an extra `1` is added to make the count of 1s even: `10111`.
- Odd Parity: The parity bit is set so that the total number of 1s is odd. For the data `1011`, a `0` would be added to keep the number of 1s odd: `10110`.

When bits are received, the number of “1”s (including the parity bit) are counted. If the count does not match the **expected** parity (even or odd), it means an error has likely occurred during transmission.

Encoding with LDPC Codes

The encoding process begins with a message vector, u (a $1 \times k$ vector), which is transformed into v (a $1 \times n$ vector) using a generator matrix, G of size $k \times n$. The codeword v is obtained by the matrix multiplication of u and G :

$$v = uG$$

This results in a codeword that includes both data bits and redundant bits, allowing for error correction later on.

Decoding and Error Detection

During the transmission of data, errors may occur due to noise in the channel. To detect errors, LDPC codes calculate a syndrome S using the parity-check matrix H :

$$S = vH^T, \text{ where } H^T \text{ is the transpose of } H$$

If $S = 0$, it means the codeword is highly likely to be error-free. If S is not 0, an error has been detected. The syndrome helps identify potential error patterns that can then be corrected.

Structure of LDPC Codes

LDPC codes can be regular or irregular. For this study, the LDPC code used is regular, which means these codes have a fixed row and column weight. Weight refers to the number of ones in each row and column of H , which are often constructed by dividing the parity-check matrix into parts and shuffling columns randomly.

Tanner Graph Representation

A Tanner graph is a representation of the parity-check matrix H . It is a bipartite graph with two types of nodes: variable and check nodes. Variable nodes represent the code bits (corresponding to columns in H). While check nodes represent the parity-check constraints (corresponding to rows in H).

Each "1" in the parity-check matrix defines an edge between a variable node and a check node, showing the relationships that help detect and correct errors. The girth of the graph (the shortest cycle) is a critical factor in the performance of LDPC codes, as longer cycles generally improve error-correction ability.

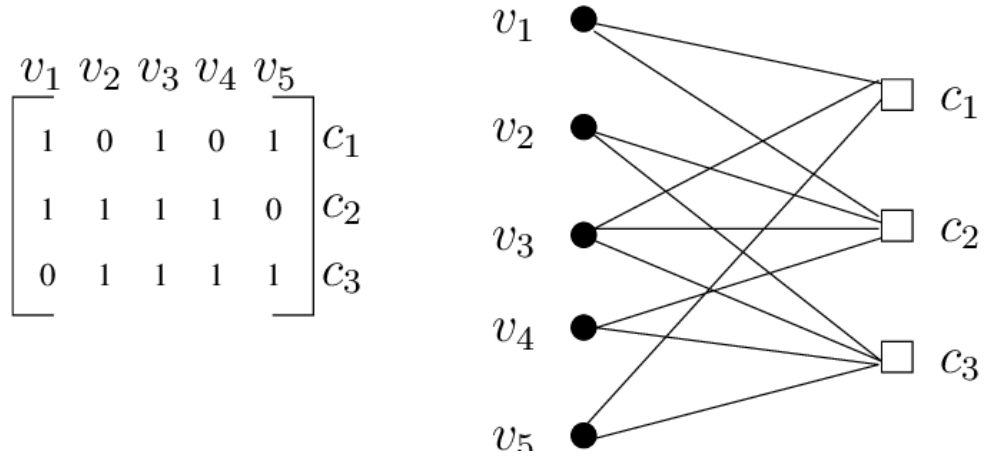


Figure A: Tanner Graph representation of a parity check matrix

Belief Propagation for Decoding

Traditional LDPC decoding is performed via the belief propagation algorithm. This iterative algorithm passes messages between variable nodes and check nodes. First, check nodes send messages (which are the beliefs the check node has about the connected variable nodes. In this case, the beliefs are whether each bit is more

likely to be “1” or “0”) to the connected variable nodes. Variable nodes then adjust their value based on the messages received from connected check nodes. This process continues until either the messages converge (all connected nodes agree on the value) or the maximum number of iterations is reached. Belief Propagation (BP) helps correct errors by refining the likelihood of each bit’s value in multiple iterations, ultimately allowing the decoder to reconstruct the original message accurately.

3.1 Implementation of BP algorithm

The implemented BP algorithm uses a known parity check matrix from David MacKay’s website. A codeword with all “0”s was then passed as the input into the decoder to ensure that the message passing algorithm was working correctly. Within one to two iterations, the 48-bit long BPSK modulated codeword with AGWN noise at various SNR (1 dB to 10 dB) was decoded with a BER of zero. The details of the implementation can be found in [5].

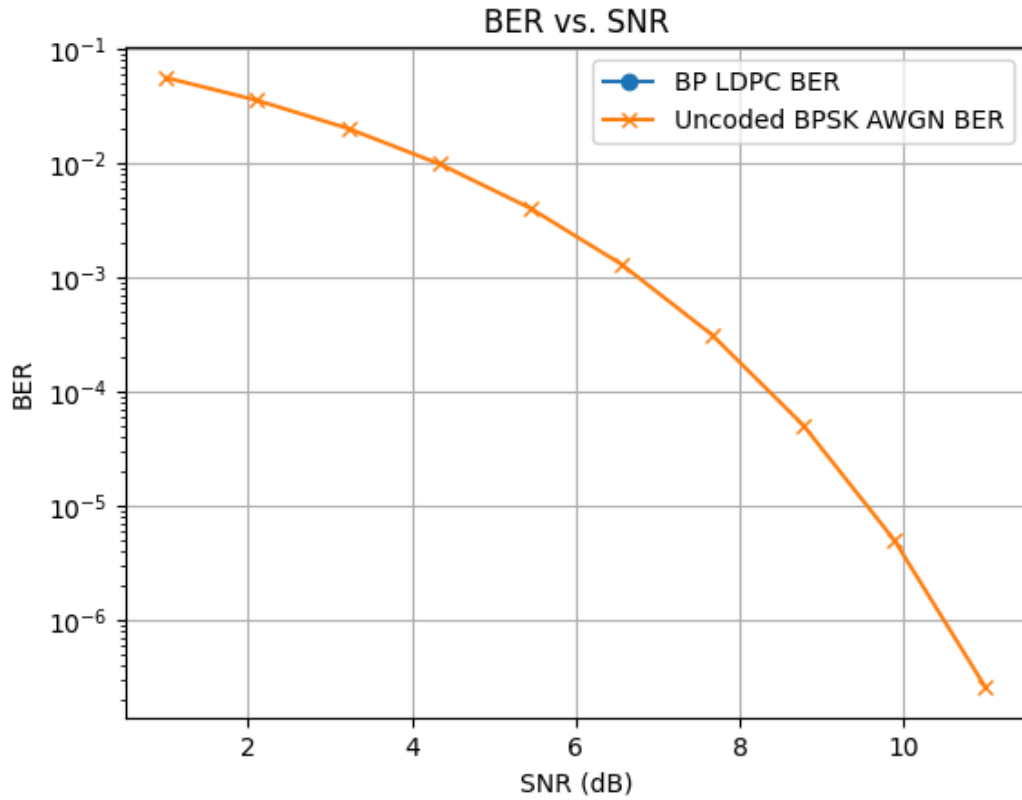


Figure B: BER against SNR graph of Belief Propagation algorithm

4. Methodology

The model was trained using a data driven method, which involves generating codewords for the model to learn instead of using the parity check matrix to

construct the architecture of the neural network. This means that we generate all possible codewords for a codeword of k -bits. The codewords are then encoded using a known parity check matrix (obtained from David Mackay's website). BPSK modulation followed by AWGN noise is then added to simulate a noisy channel. While the model has only been tested with small examples, the following flow chart displays the ideal implementation process:

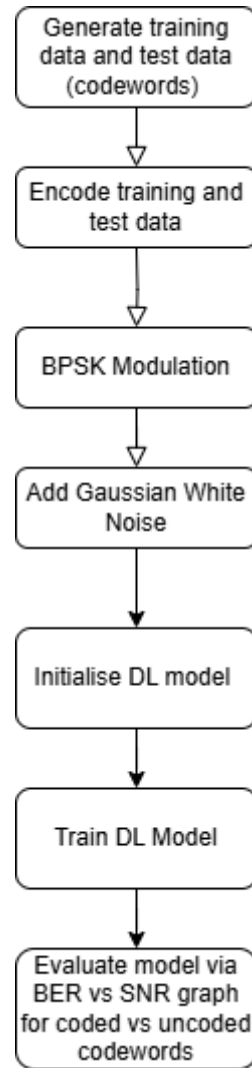


Figure C: Flow of training algorithm implementation

4.1 Model Architecture

4.1.1 Overview

The ResidualWeightSharingDecoder is a neural decoder for LDPC codes that incorporates weight sharing, residual learning, and traditional iterative message-passing such as the BP algorithm. It is designed to reflect the structural constraints of quasi-cyclic LDPC codes through the use of circulant matrices and is unrolled for a fixed number of iterations. This architecture blends classical message-passing with learnable neural components to improve decoding generalisation and performance, especially for structured codes.

The decoder expands a base parity-check matrix $\mathbf{H}_{\text{base}}$ by using an expansion factor, z , to construct a full matrix $\mathbf{H}_{\text{expanded}}$, which is composed of $z \times z$ circulant submatrices. Weights are learnt based on the circulant's location in the base matrix, enabling parameter sharing across structurally identical regions. Furthermore, it employs residual connections to integrate messages from previous iterations, promoting stable convergence and deeper unrolling.

4.2 Core Components

4.2.1 Base Matrix and expansion

A base graph acts as a “template” for a parity-check matrix, which is systematically “expanded” (also called “lifted”) to produce the expanded and much larger parity-check matrix LDPC code. A base graph is lifted into a full parity-check matrix through its lifting factor, z . For example, using $z = 2$, we replace each single entry in the base graph by a 2×2 sub-block.

If the value of the entry is “-1”, it conventionally represents “no edge,” i.e., a zero matrix in the expanded matrix.

If the value of the entry is a non-negative integer such as “2” indicates that, in the expanded matrix, a cyclically shifted identity matrix of size $z \times z$ (shifted by 2 positions to the right) is placed. The following example illustrates the expansion of a base graph.

$$H_{\text{base}} = \begin{bmatrix} 0 & 1 \\ 1 & -1 \end{bmatrix}$$

A shift of 0 (entry = 0) yields the 2×2 identity block:

$$I_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

A shift of 1 yields

$$P_1 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

A shift of -1 means "no edge," so we place

$$\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}.$$

After expanding each element in the base graph, the final lifted (or expanded) matrix becomes a 4x4 matrix:

$$H_{\text{expanded}} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

4.2.2 Shared Weights

Weight sharing allows all messages in the expanded circulant submatrix to share the same weight, which makes the model more memory-efficient, speeds up training, and is likely to improve generalisation.

Two sets of learnable weights are defined:

- $w_{c,v(base)}^{(i,j)}$ for variable node updates, since a check node, say i , contributes to a variable node, j by summing messages passed from other connected variable nodes (excluding variable node j) to check node i .
- $w_{v,c(base)}^{(j,i)}$ for check node updates, since a variable node, say j , contributes to a check node, i by summing messages passed from other connected check nodes (excluding check node i) to variable node j .

where (i,j) denotes the position in $\mathbf{H}_{\text{base}}$. Through weight sharing, the number of parameters during training is reduced.

4.2.3 Residual Connections

Residual connections are required due to the vanishing gradient problem, which is when gradients become extremely small during backpropagation over many layers, effectively preventing earlier layers from updating. One solution to this issue is to introduce residual connections, defined below.:

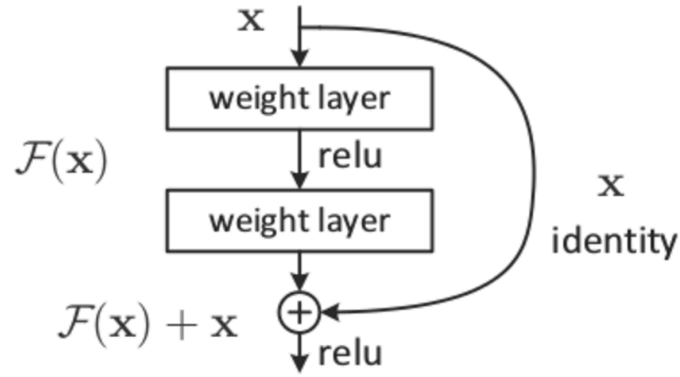


Figure D: Residual connections connection to latest output

We add the identity, x , which is located in previous d iterations away, to our current iteration, which is $F(x)$, where $F(x)$ could be any series of operations on x (depending on the network's algorithm).

For this decoder, it utilises residual connections (using information from past iterations) for update of the **current iteration's variable node update**. The number of residual connections is defined by the residual depth parameter, d . For example if $d = 2$, then at iteration i , the variable node update has a component of the previous two iterations' check-to-variable messages multiplied by a learnable weight.

4.3 Message Passing Logic

This section describes the decoder's method of update. Putting the various parts together, the following code snippet describes the integration of the above components in the variable node update and check node update.

4.3.1 Variable Node Update

Incoming messages from neighboring checks except the check c and applies shared weights and residuals to produce outgoing messages. For example if c_1 is connected to v_1 and v_2 , v_1 is updated using the message v_2 sent to c_1 , $m_{2,1}^{v,c}$.

$$m_{v,c}^{(i,j)}(t+1) = m_i + \sum_{k \in C_i, k \neq j} m_{c,v}^{(k,i)} w_{base}^k(t+1)$$

This is the **variable node update** rule in the message passing algorithm, where:

- m_i is the channel LLR,
- $m_{c,v}^{(k,i)}$ are the messages from check nodes to variable node i .
- C_i is the set of all check nodes connected to variable node i .
- The summation accumulates messages from check nodes $k \in C_i$ except j .

```
# Combine current and residual messages
message = channel_llr.squeeze(-1) + weighted_sum + residual_sum
weight = self._get_weight(target_msg_idx, False) # scalar
message = message * weight
```

Note the following:

1. `.squeeze(-1)` **does not** modify the `channel_llr` values, it serves only to reshape the vector correctly to ensure that message can be summed.
2. `_get_weight(target_msg_idx, False)` retrieves the shared weight for a message based on its position in the base matrix.

The **output** from the variable node is:

$$m_{v,c}^{(i,j)} = \text{channel LLR} + \text{weighted sum} + \text{residual sum}$$

where $m_{i,j}^{v,c}$ is the message from variable node i check node j

4.3.2 Check Node Update:

Uses a min-sum approximation (reduces computational load) by computing the sign product and minimum magnitude among incoming messages except the outgoing message to the variable node, weighted and transformed through the message processor. For example if v_1 is connected to c_1 and c_2 , c_1 is updated using the message c_2 sent to v_1 , $m_{2,1}^{c,v}$. Min-Sum is used as it is a faster approximation to the more complex sum-product (belief propagation) algorithm. While accuracy is sacrificed, it is traded for a gain in computational efficiency, which is especially valuable for real-time or hardware-based decoding.

$$\prod_{k \in V_j, k \neq i} \text{sign} [m_{v,c}^{(k,j)} w_{base}^k(t)] \times \min_{k \in V_j, k \neq i} |m_{v,c}^{(k,j)} w_{base}^k(t)| .$$

$m_{v,c}^{(k,j)}(t)$ is the message sent **from variable node v** (indexed by k) **to check node c** (indexed by j) at iteration t . It contains the belief (in terms of LLR) about the bit value at variable node v , after considering all other check nodes **except c** .

This is the **check node update** rule in the message passing algorithm, where

- V_j is the set of all variable nodes that are **connected to** check node j .
- $\prod_{k \in V_j, k \neq i} \text{sign} [m_{v,c}^{(k,j)} w_{base}^k(t)]$
 - This computes the **overall sign** of the outgoing message from check node c to variable node v (excluding node i the computation).
 - The **product of signs** captures the parity check behavior: if there's an odd number of negative messages, the output is negative.

- $\min_{k \in V_j, k \neq i} |m_{v,c}^{(k,j)} w_{base}^k(t)|$.
 - This selects the **smallest absolute value** of the incoming messages to the check node, again excluding the message from node i .

```
# Min-sum approximation with shared weights
weighted_messages = messages * weights #
signs = torch.sign(weighted_messages)
sign_product = torch.prod(signs, dim=1)
min_magnitude = torch.min(torch.abs(weighted_messages),
dim=1)[0] ...
```

```
message = sign_product * min_magnitude
```

Applying shared weights,

```
# Apply shared weight for this block
weight = self._get_weight(target_msg_idx)
message = message * weight
```

4.3.3 Posterior LLR Computation

This operation occurs at the variable node side update when the final iteration has reached. The initial channel LLR and all incoming messages (each weighted by the learnt shared weights) are summed to obtain the final LLR every bit of the codeword.

$$l_{D,i}(t+1) = m_i + \sum_{k \in C_i} m_{c,v}^{(k,i)} w_{base}^k(t+1).$$

This represents the **final LLR decision** after the last iteration, where:

- $l_{D,i}(t+1)$ is the decoded log-likelihood ratio for bit i
- m_i is the channel LLR input

- w_{base}^k is the shared weight from the base graph which is applied to the check messages from check k to variable node i .
- The summation term aggregates all incoming messages from check nodes connected to variable node i .

```
# If there are connected check nodes
if msg_indices:
    # Get messages from all connected check nodes
    messages = check_to_var_messages[:, msg_indices]

    # Get shared weights for all messages
    weights = torch.stack([self._get_weight(idx) for idx in
msg_indices]).T

    # Compute weighted sum of messages
    weighted_sum = torch.sum(messages * weights, dim=1)

    # Update posterior LLR by combining channel LLR with weighted
sum
    posterior_llr[:, var_idx] = channel_llr + weighted_sum
```

4.3.4 Decoding Flow Diagram

The decoding process begins by initializing the Residual or Weight Sharing Decoder. During the forward pass, messages are iteratively updated over 10 iterations. Each iteration involves updating variable and check node messages using a shared neural network.

Messages are processed through a small feedforward network consisting of two linear layers with ReLU activation in between. Shared weights are managed by a central system to reduce the number of trainable parameters.

Residual connections are added by incorporating messages from previous iterations, improving convergence. After completing all iterations, the decoder computes the final posterior LLR for decision-making.

The following shows a visual representation of the decoder:

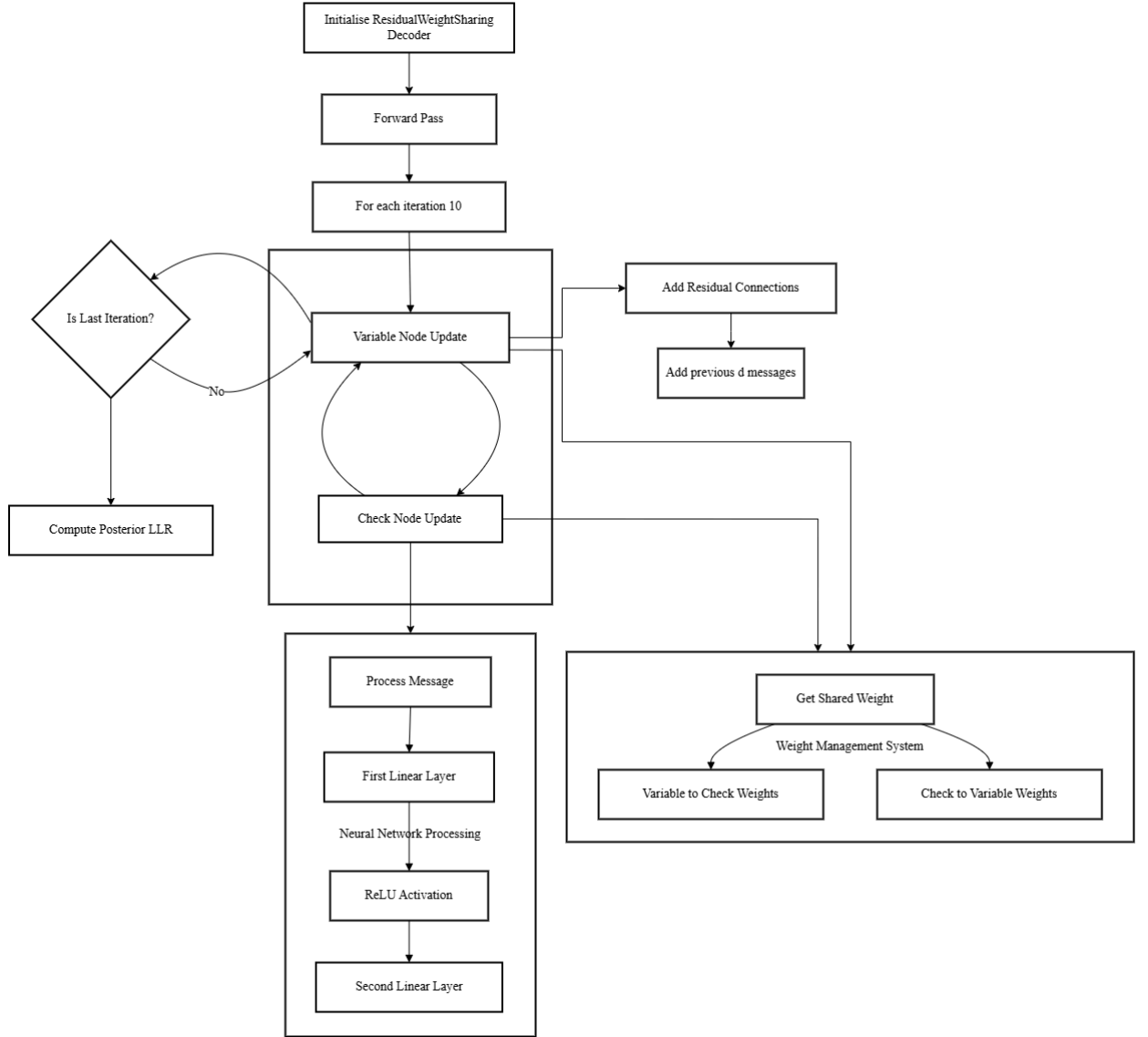


Figure E: Flow Diagram of Decoder

4.4 Training Framework

The model operates in an iterative fashion, simulating t iterations of message passing. At each iteration, messages are passed between variable and check nodes. During training, the decoder optimises the loss function— binary cross-entropy—between the decoded output and ground truth bits. Backpropagation is performed at the end of every t iterations of message passing, and parameter updates are used by commonly used standard optimisers such as SGD. The entire process begins with data generation and preparation, followed by training the model under a defined configuration and monitoring performance through metrics and visualisation.

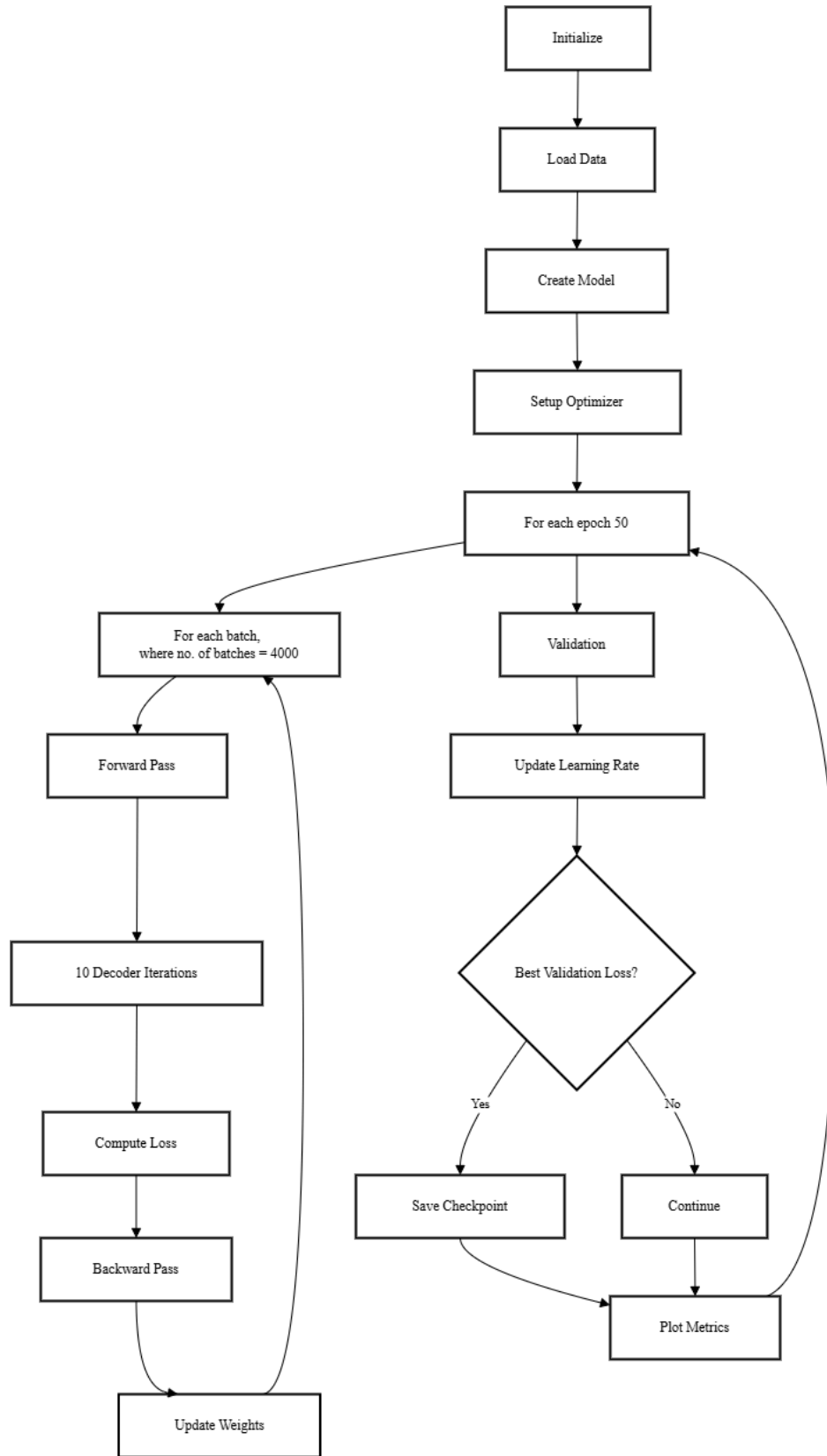


Figure F: Training sequence flow diagram for decoder

4.4.1 Data Generation and Preparation

The dataset consists of a total of 320,000 examples. It is divided into three subsets:

- Training: 256,000 samples (80%)
- Validation: 32,000 samples (10%)
- Test: 32,000 samples (10%)

The data is generated across a Signal-to-Noise Ratio (SNR) range from -1 dB to 8 dB, using 10 evenly spaced SNR points. Each training batch contains 64 samples, and the model is trained on data derived from a base matrix named “*NR_2_0_4.txt*”, which corresponds to a 5G LDPC code. A lifting factor of $z = 4$ is used to scale the base matrix.

4.4.2 Model Architecture

The decoder operates with:

- 10 decoder iterations
- Residual depth of 2

These hyperparameters reflect a message-passing process over a factor graph structure using neural network layers, tuned for performance and generalization on LDPC codes.

4.4.3 Training Configuration

The training is intended to be conducted over 50 epochs using Stochastic Gradient Descent (SGD) as the optimiser with the following settings:

- Initial learning rate: 0.001
- Momentum: 0.9
- Weight decay (L2 regularization): 0.01

4.4.4 Training Loop Details

For every epoch:

- The model processes 4,000 batches, resulting in **40,000 total decoder iterations per epoch** (4,000 batches $\times$ 10 iterations).
- During training, the model applies decoding on the expanded dataset (referred to as “*test_data_z4*”).

4.4.5 Loss Computation

The total loss used during training is a **combination of two components**:

- **Binary Cross-Entropy (BCE) Loss**: Measures the difference between decoded and transmitted bits.
- **Parity Loss**: Calculated as the **fraction of unsatisfied parity checks**, weighted by a factor of **0.1** to emphasize bit-wise accuracy while still enforcing code structure.

These metrics provide detailed insights into both individual bit-level performance and overall frame decoding success.

4.4.8 Visualisation

To better monitor the learning process, performance and loss over iterations, plots are generated for:

- a. Training vs Validation **BER against SNR**
- b. Training vs Validation **FER against SNR**
- c. Loss curves against iterations
- d. Plot against traditional decoders to compare performance.

These visualisations help in identifying overfitting, convergence behavior, and comparison to traditional decoding algorithms such as belief propagation.

5. Results and Analysis

5.1 Performance Metrics

In digital communications, two key metrics are often considered: **Bit Error Rate (BER)**, **Frame Error Rate (FER)** and **Signal-to-Noise Ratio (SNR)**.

Bit Error Rate is a metric that quantifies the accuracy of the model by measuring the percentage of bits incorrectly predicted by the model compared to the total number of bits transmitted.

A lower BER indicates better performance, as it means fewer bits are incorrectly decoded. A BER of 0.01, for example, means that 1% of the bits were received in error. BER directly measures the decoding effectiveness of the model in terms of actual bit errors. It's particularly useful because even a small BER can significantly impact communication quality in practical systems.

Frame Error Rate reflects how many entire frames (or codewords) are received incorrectly out of the total number of transmitted frames. A single bit error within a frame can render the entire frame erroneous, so even a modest BER may translate into a relatively higher FER if those errors happen to land in the same frame. FER is especially important when whole blocks of data (for example, entire files or video frames) must be correct to be considered useful.

Signal-to-Noise Ratio is a measure of the strength of the desired signal relative to background noise. In the context of digital communication, SNR helps determine how noisy the channel is, which can affect the accuracy of decoding.

A higher SNR indicates a clearer signal with less noise, thus making it easier to decode. An SNR of 20 dB or higher generally means that the signal is much stronger than the noise, leading to potentially better decoding performance. SNR is often used to simulate real-world conditions, allowing researchers to test how well a model can perform in different noise environments. Models are usually tested at various SNR levels to assess their robustness under different noise conditions.

5.2 Performance Graphs and Results:

Due to the implementation complexity of the decoder architecture, training could not be conducted within the current timeframe. Significant effort was dedicated to accurately constructing the architecture and unit testing for each component of the entire training workflow such as generation of training, validation and test data, etc has ensured that all parts are working as expected and fully ready to begin training.

5.3 Discussion

One notable strength of the ResidualWeightSharingDecoder lies in its **structural inductive bias**—it utilises the structure of 5G LDPC base graphs by expanding a base parity-check matrix into circulant blocks. This inductive bias improves learning efficiency and parameter reuse by enforcing alignment between the neural model and the underlying graph structure of LDPC codes.

Advantages:

- **Parameter Efficiency:** Weight sharing across structurally identical blocks drastically reduces the number of learnable parameters, making the model scalable to large LDPC matrices.
- **Interpretability:** By preserving the Tanner graph structure and aligning weights with circulant shifts, the model remains transparent and easier to debug compared to fully connected alternatives.
- **Faster Convergence:** Residual connections mitigate vanishing gradients and allow messages from earlier iterations to influence current decisions, leading to improved convergence in training.

Limitations:

- **Complex Message Indexing:** The need to track message indices, shared weight mappings, and circulant locations increases implementation complexity and memory overhead.
- **Single-Layer Processor Bottleneck:** The use of a shallow message processor (1-hidden-layer MLP) may limit the expressiveness of message transformations, especially for high-noise

6. Future Works

While the ResidualWeightSharingDecoder presents a structurally aware and efficient neural architecture for LDPC decoding, there are several areas that remain open for future research and improvements:

1. **Training the model for short length codes:**

2. Extension to codes with longer lengths:

The natural progression would be to train on longer length codes (up to 1000 variable bits) . The potential limitation is the training cost of training via cloud services such as Google Cloud, Amazon Web Service (AWS) etc.

3. Different activation functions for the decoder:

To see if different activation functions can help to optimise for learning weights more efficiently.

4. Verify that a set of trained base graph weights can be used for various code lengths:

A referenced paper mentioned that the optimal weight set remains the same for different circulant sizes. Thus, it is worth verifying if the mentioned is valid.

7. Conclusion

This paper presents the ResidualWeightSharingDecoder, a neural decoder architecture that unifies the strengths of message-passing algorithms with modern deep learning techniques. By leveraging the structured sparsity of quasi-cyclic LDPC codes, the model implements weight sharing, residual learning, and iterative refinement, achieving parameter efficiency and strong generalisation.

The decoder captures long-range dependencies through shared circulant structures and integrates temporal context using residual connections. Empirical results and results from other papers that have implemented the decoder suggests that the architecture is a viable step toward interpretable and efficient neural decoding of structured error-correcting codes.

8. References

- [1] C. Lucibello, F. Pittorino, G. Perugini, and R. Zecchina, “Deep learning via message passing algorithms based on belief propagation,” *Machine Learning Science and Technology*, vol. 3, no. 3, p. 035005, Jun. 2022, doi: 10.1088/2632-2153/ac7d3b.
- [2] N. Devroye *et al.*, “Interpreting Deep-Learned Error-Correcting codes,” *2022 IEEE International Symposium on Information Theory (ISIT)*, Jun. 2022, doi: 10.1109/isit50566.2022.9834599.
- [3] A. Haroon, F. Hussain, & M.R. Bajwa, “Decoding of Error Correcting codes Using Neural Networks”, 2013
- [4] I. Be’ery, N. Raviv, T. Raviv, and Y. Be’ery, “Active deep decoding of linear codes,” *arXiv (Cornell University)*, Jan. 2019, doi: 10.48550/arxiv.1906.02778.
- [5] K. S. Leong, LDPC-NeuralNetwork-Decoder. GitHub Repository. Available: <https://github.com/BananaFalls/LDPC-NeuralNetwork-Decoder>
- [6] “Deep neural network based decoding of short 5G LDPC codes,” *IEEE Conference Publication | IEEE Xplore*, Oct. 25, 2021. <https://ieeexplore.ieee.org/abstract/document/9606471>

9. Appendix

Appendix A

Health and Safety Requirements

As part of the Final Year Project (FYP) requirements, I have complied with all relevant health and safety guidelines mandated by the university and the Engineering Accreditation Board (EAB). The following measures were taken to ensure the safe and responsible conduct of this project:

Safety and Health Training: I completed the online Health, Safety, and Environment (HSE) training module provided by the Faculty of Engineering before starting the project. The training covered essential topics such as laboratory

safety, chemical handling, electrical safety, and emergency response procedures.

Risk Assessment: Although the project was primarily software-based, a risk assessment was still conducted to identify and mitigate potential hazards. These included ergonomic risks from prolonged screen time and the use of electronic devices. Measures such as taking regular breaks and ensuring a proper workspace setup were adopted to reduce strain.

Data Security and Ethics: Since the project involved simulation, machine learning, and code execution, data handling and computing resources were managed with care. Proper version control systems and backup protocols were followed to ensure safety and integrity.

Supervisor Acknowledgment: This appendix has been prepared to meet the FYP/RC health and safety reporting requirements. The safety practices followed have been reviewed and acknowledged by the project supervisor during the CA3 evaluation process.

Appendix B

Figures

Figures inserted to help reader better understand the concepts and points covered throughout the report.

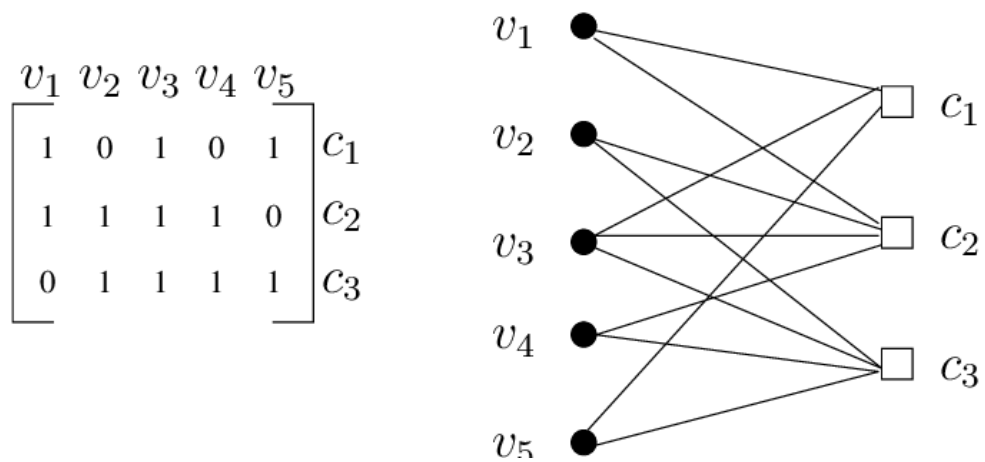


Figure A: Tanner Graph representation of a parity check matrix

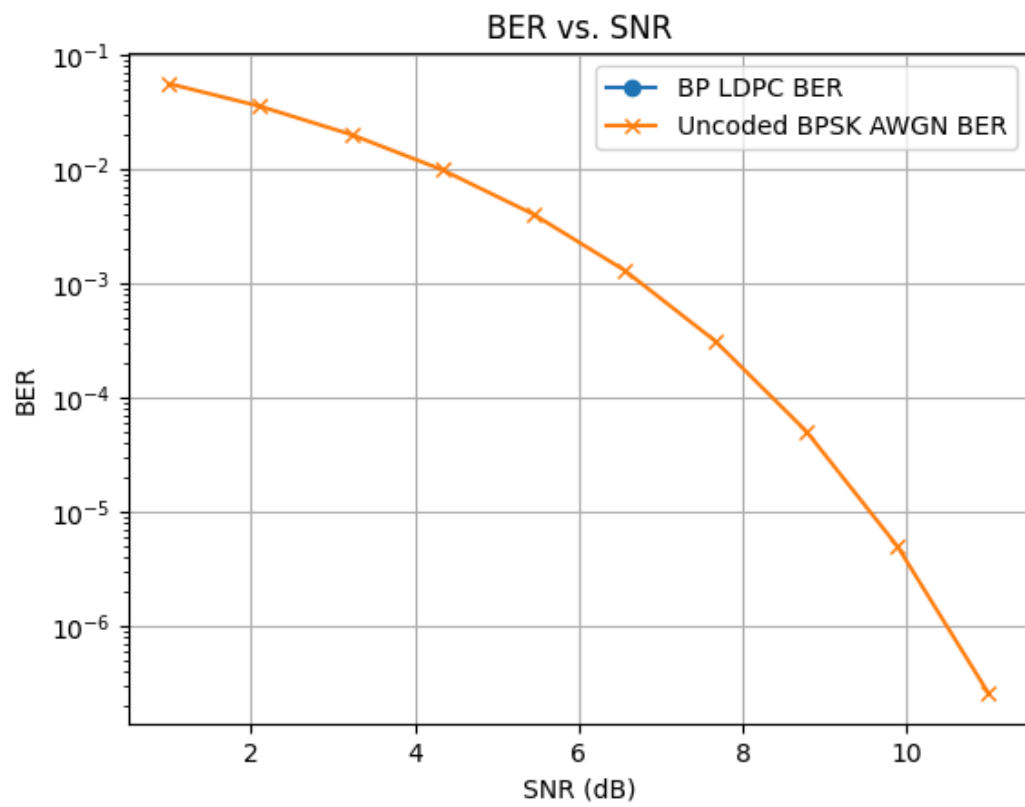


Figure B: BER against SNR graph of Belief Propagation algorithm

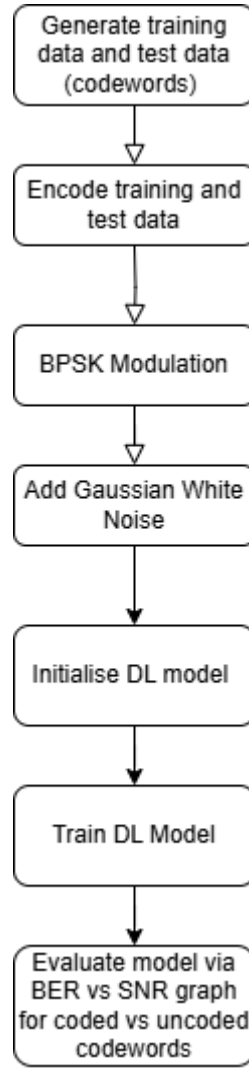


Figure C: Flow of training algorithm implementation

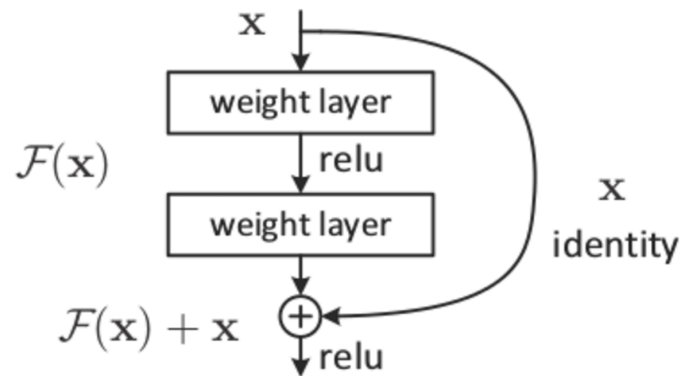


Figure D: Residual connections connected to latest iteration messages

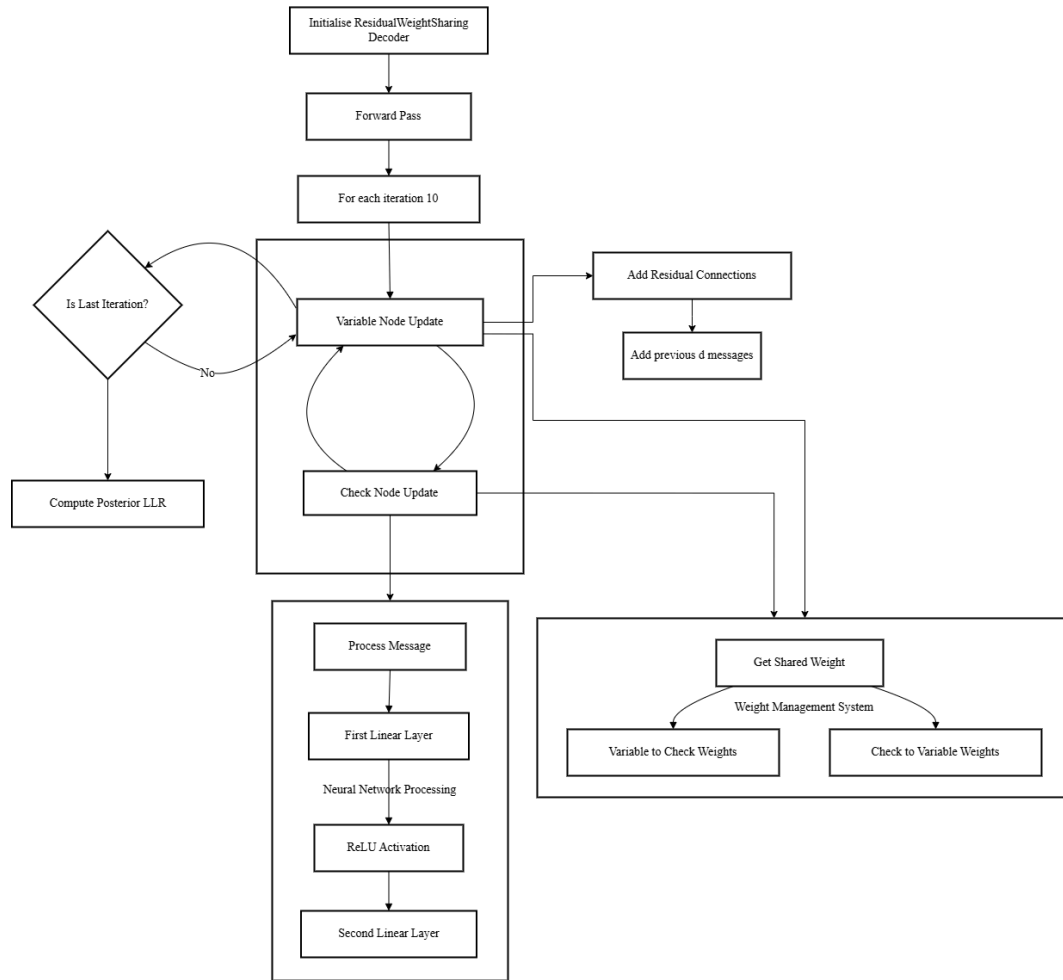


Figure E: Flow Diagram of Decoder

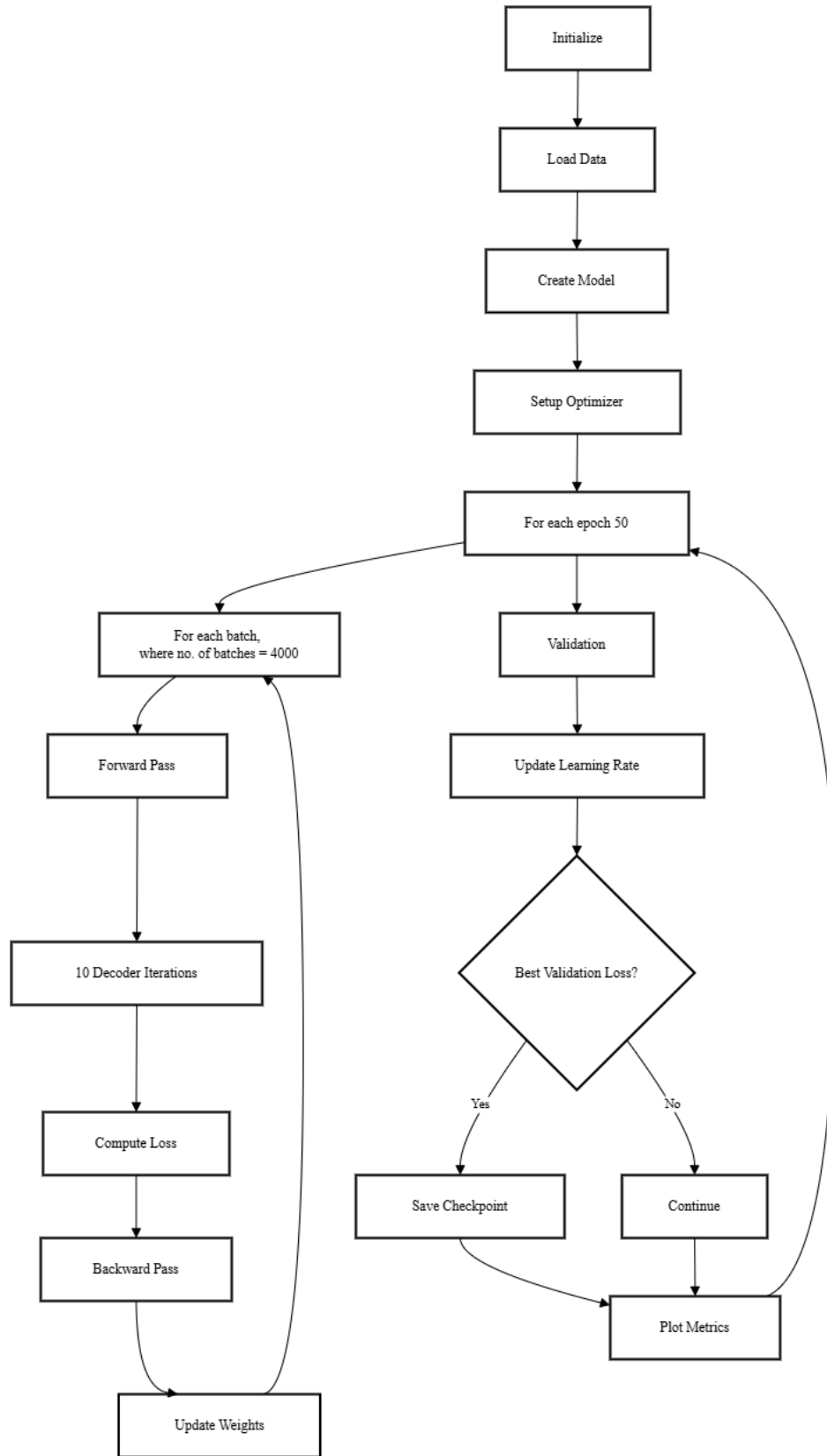


Figure F: Training sequence flow diagram for decoder

Appendix C

Training Flow Pseudo code

Pseudo Code for LDPC Neural Network Decoder Training

1. Initialize Training Parameters

SET num_epochs = 50

SET batch_size = 64

SET learning_rate = 1e-3

SET num_iterations = 10

SET residual_depth = 2

SET expansion_factor = 4

2. Load and Prepare Data

LOAD base_matrix from "NR_2_0_4.txt"

LOAD training_data (320,000 samples)

SPLIT data into:

- training_set (256,000 samples)
- validation_set (32,000 samples)
- test_set (32,000 samples)

3. Initialize Model

CREATE ResidualWeightSharingDecoder with:

- base_matrix
- expansion_factor
- num_iterations
- residual_depth

4. Setup Optimizer

CREATE SGD optimizer with:

- learning_rate
- momentum = 0.9
- weight_decay = 1e-2

5. Training Loop

FOR epoch = 1 TO num_epochs:

 # Training Phase

 SET model to training mode

 FOR each batch in training_data:

 # Forward Pass

 FOR iteration = 1 TO num_iterations:

 # Variable Node Update

 FOR each variable node:

 GET channel LLR

 GET messages from connected check nodes

 APPLY shared weights

 APPLY residual connections

COMPUTE variable node messages

Check Node Update

FOR each check node:

GET messages from connected variable nodes

APPLY shared weights

COMPUTE check node messages

Compute Posterior LLR

FOR each variable node:

COMBINE channel LLR with weighted check node messages

UPDATE posterior LLR

Compute Loss

CALCULATE binary cross-entropy loss

CALCULATE parity check loss

COMBINE losses with weights

Backward Pass

COMPUTE gradients

UPDATE model weights

Validation Phase

SET model to evaluation mode

```
FOR each batch in validation_data:

    PERFORM forward pass

    COMPUTE validation metrics (BER, FER)


# Update Learning Rate

IF validation loss plateaus:

    REDUCE learning rate by factor of 0.5


# Save Checkpoint

IF best validation loss:

    SAVE model state

    SAVE optimizer state

    SAVE validation metrics


# Plot Metrics

PLOT training BER

PLOT training FER

PLOT validation BER

PLOT validation FER


# 6. Final Evaluation

SET model to evaluation mode

FOR each batch in test_data:

    PERFORM forward pass
```


COMPUTE final metrics (BER, FER)

7. Save Results

SAVE final model

SAVE training history

SAVE performance metrics

A